

PDBlend: online routing, instance assignment, and execution

Current homogeneous PDBlend runner | source snapshot: 24 September 2026

Prefill

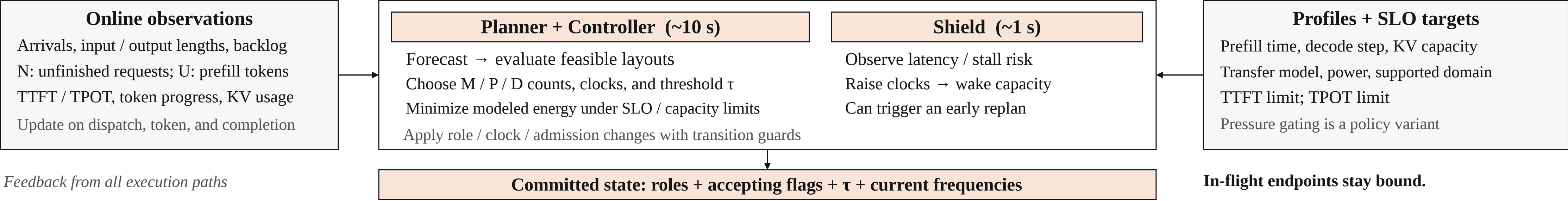
Decode

Wait / KV transfer

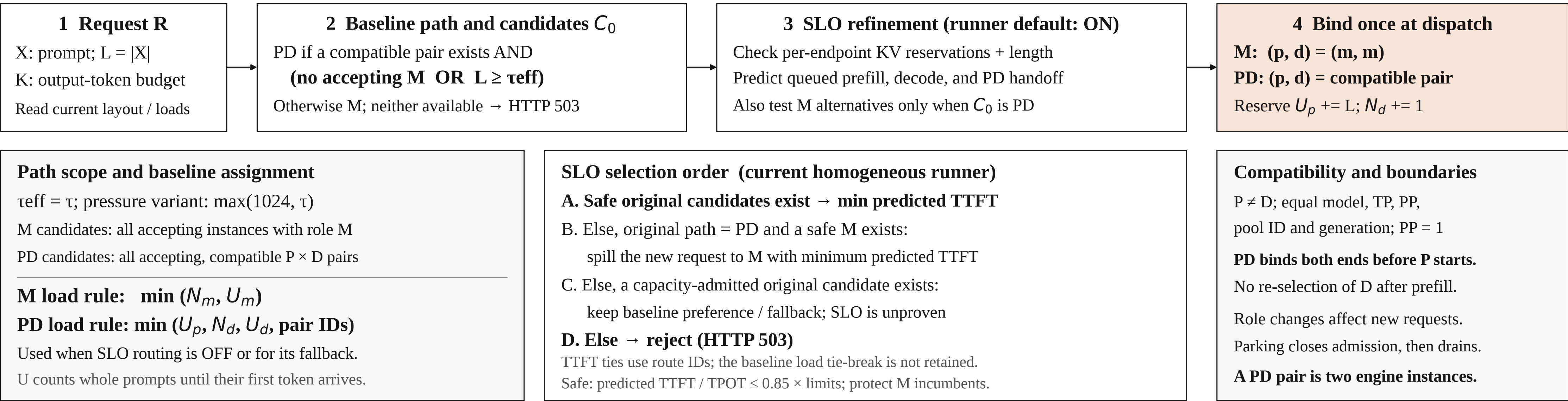
Output token

Control

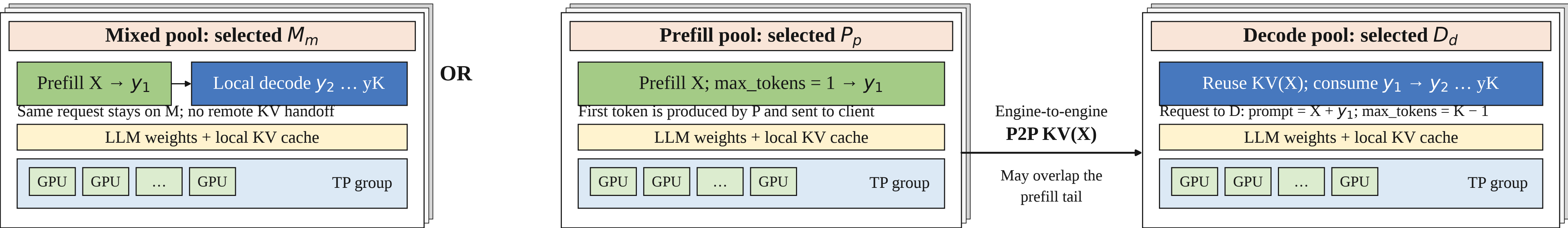
(a) Two control timescales publish the layout used by every new request



(b) Per-request decision: select a path, evaluate candidates, then bind (prefill, decode)



(c) Executing the assigned request: one Mixed engine, or two separate P / D engines



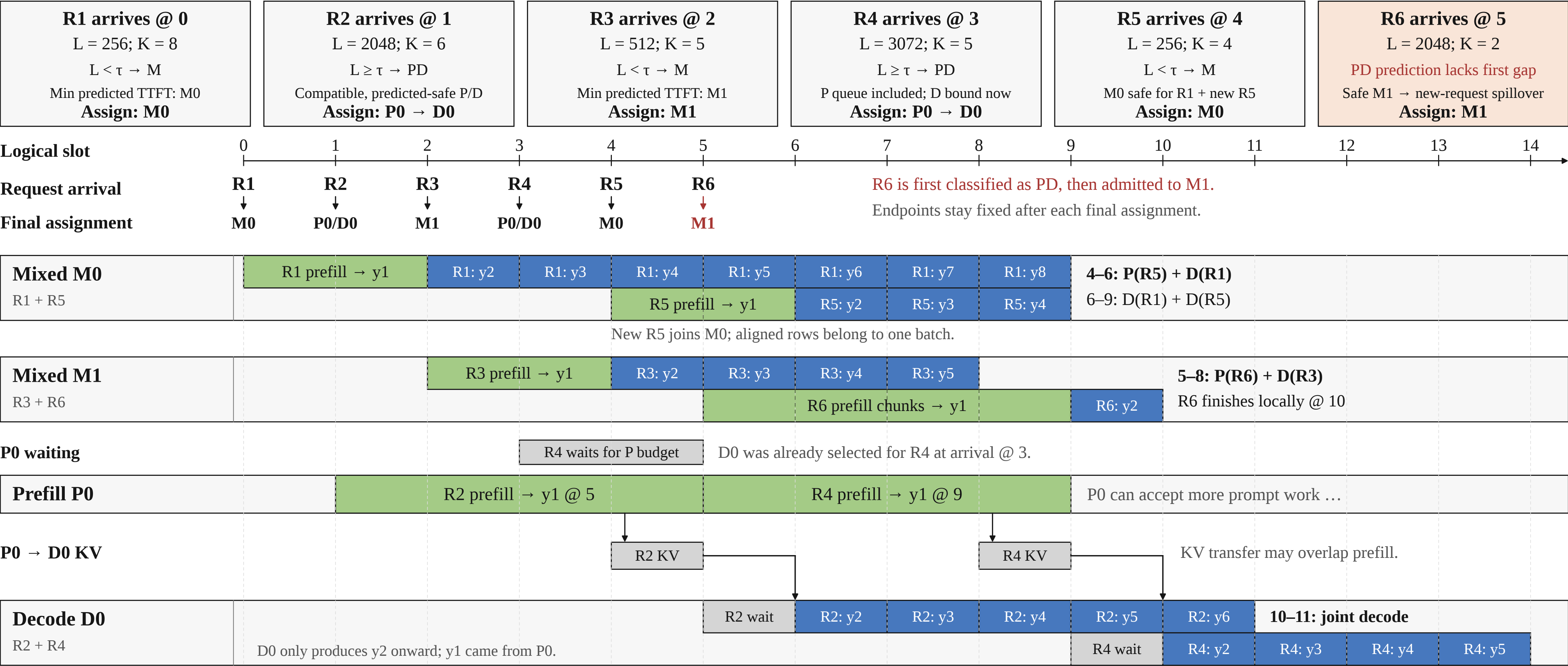
Inside each engine: native vLLM V1 continuous batching; running work first, then eligible waiting work (default FCFS).

Mixed batches may contain prefill chunks and decode tokens. On first token: $U_p -= L$. On confirmed completion: $N_d -= 1$.

(d) Six arriving requests: route decisions → per-instance batches → per-request output tokens

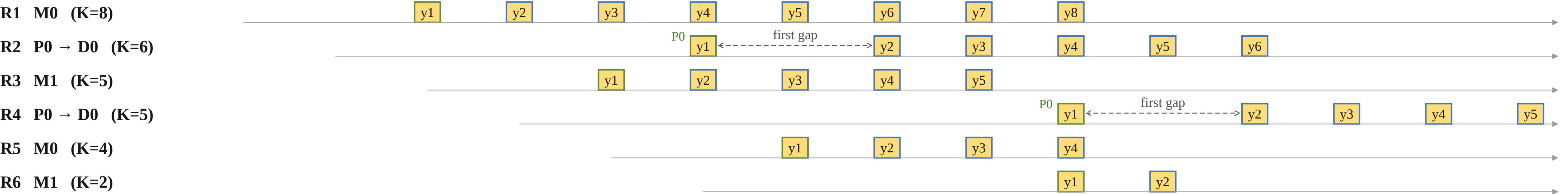
Fixed layout: M0–M3 + P0 + D0; $\tau = 1024$; SLO routing ON. Only M0/M1 are expanded; M2/M3 serve background work.

The shown safe candidates / TTFT rankings are assumed. Logical slots and bar widths are schematic, not measured or simulated.



Same-column blocks within one instance = one continuous batch; M and PD instances can execute at the same time.

Client outputs per request (30 tokens total): green outline = first token; blue outline = subsequent decode token



R2: TTFT 1 → 5; first gap 5 → 7. R4: TTFT 3 → 9 includes P waiting; first gap 9 → 11. P produces 2 tokens; D produces 9.

R6: missing PD first-gap coverage alone does not force M; here M1 is assumed capacity-safe, SLO-safe, incumbent-safe, and best by TTFT.

M2/M3 background load makes them less preferred in this example. R4 waits because earlier P work occupies the available scheduling budget.

K is fully generated in this example. All times are logical slots; no measured latency, throughput, or profile qualification is claimed.